WHITE PAPER

Post-Symbolic Runtime and SDPF

A Formal Methodology for High-Assurance AI-Governed Autonomous Systems

Implications for Autonomous Vehicles · Spacecraft · Power Grids · Medical Equipment

Author	**Hamza Abdullah**
Role	Architect and Principal Researcher — SDPF and PSR
Framework	Software Development Prompting Framework (SDPF) v1.3.1
Architecture	Post-Symbolic Runtime (PSR) — PSR = (S, M, P, T, A, R)
Reference Impl.	Industrial Controller PSR — Style 14 · 27 Requirements · 56 Tests · CLOSURE STATUS: COMPLETE
Date	2026

> *The specification is the work. The model executes it. Fix everything before running code.*
>
> *— Hamza Abdullah*

ABSTRACT

This white paper presents the Post-Symbolic Runtime (PSR) and the Software Development Prompting Framework (SDPF) as a unified formal methodology for producing high-assurance autonomous systems. The paper argues that the dominant failure mode in AI-assisted software development is not model inadequacy but specification inadequacy, and that a complete, verified specification — combined with a formally defined mission-persistent runtime architecture — produces correct, auditable, and certifiable systems as a natural output of the development process. The implications for safety-critical domains including autonomous vehicles, spacecraft, power grids, and medical equipment are examined in detail. A complete reference implementation — the Industrial Controller PSR — is presented as evidence of concept, achieving 56 passing tests against a locked specification under SDPF Style 14 Dynamic Criticality Extension.

Contents

1. The Problem: Specification-Induced Failure 3
2. The Bounded Stochasticity Theorem 5
3. The Software Development Prompting Framework (SDPF) 6
4. The Post-Symbolic Runtime (PSR) 9
5. SDPF and PSR: A Unified Methodology 11
6. Reference Implementation: Industrial Controller PSR 12
7. High-Assurance Domain Applications 14
8. Regulatory Alignment 17
9. The Time and Cost Argument 19
10. Conclusions and Implications 20

1. The Problem: Specification-Induced Failure

1.1 The Industry's Misdiagnosis

The artificial intelligence industry has systematically misidentified the primary failure mode of AI-assisted software development. The failure is attributed to the AI model: insufficient capability, hallucination, unreliable output. Billions of dollars are directed toward improving models. The actual diagnosis is different.

When a specification is incomplete or contains unverified technical facts, an AI model fills the gaps with its best approximation. The output looks plausible. It is not what was intended. What the industry calls hallucination is, in the overwhelming majority of practical cases, specification-induced speculation. Better models make better guesses. They are still guessing. The correct intervention is not a better model. It is a complete specification — one where there is nothing to guess.

> *The problem is not the AI model. The problem is the specification.*
>
> *— SDPF Language Specification v1.3.1*

1.2 Five Structural Failure Modes

The gap between intent and implementation is not a new problem. Software has always been built on the assumption that developers can maintain an accurate mental model of what a system should do across weeks, months, and team changes. That assumption has always been fragile. With AI as an executor, it becomes untenable.

	Failure Mode	Consequence in High-Assurance Context
1	Requirements exist only informally	No auditable basis for system behaviour. Certification impossible. Any change may introduce undetected regression.
2	Tests written after implementation	Tests verify what code does, not what it should do. Bugs are preserved in the test suite. Coverage metrics are meaningless.
3	Changes made	Silent propagation of requirement violations. In safety-critical systems, this is

	without consequence tracking	a latent hazard with no visible warning.
4	Code has no traceable justification	No safety case is possible. Regulatory bodies cannot accept systems where behaviour cannot be traced to a verified requirement.
5	Technical facts asserted but never verified	System built against wrong version numbers, deprecated APIs, incorrect paths. Discovered at deployment in the target environment.

1.3 Why Existing Solutions Are Insufficient

The industry has attempted to address these failures with issue trackers, documentation tools, API specification tools, testing frameworks, and static analysis tools. None enforce the relationship between a requirement and its implementation. They operate on code in isolation. The gap remains open.

In high-assurance domains, the consequences of this gap are not merely technical. A medical device that cannot prove its behaviour matches its specification is a regulatory liability. An avionics system without complete requirements traceability cannot achieve DO-178C certification. A power grid controller without auditable change tracking violates NERC CIP. The gap between intent and implementation is not an engineering inconvenience in these domains. It is a safety failure.

THE GAP: What is missing is not a better tool. It is a missing contract — a formal, enforceable agreement between what a system is supposed to do and what it is permitted to do, with a tamper-evident audit trail proving the two are the same.

2. The Bounded Stochasticity Theorem

Before presenting the methodology, it is necessary to resolve a fundamental apparent contradiction: SDPF is deterministic, but its executor — an AI model — is stochastic. If the same prompt submitted twice does not guarantee the same output, how can a deterministic specification produce reliable results?

2.1 The Theorem

BOUNDED STOCHASTICITY THEOREM: Let S be a complete SDPF specification defining a set of structural invariants I. Let G be a generative AI output produced from S. G satisfies S if and only if G satisfies all invariants in I. The stochastic variation in G is bounded by I — any variation that satisfies I is a conforming realisation of S.

2.2 What the Theorem States

The theorem states that AI output does not need to be deterministic to be reliable. It needs to be structurally invariant. Two implementations that differ entirely in surface form — different variable names, different code structure, different documentation style — but that both satisfy the same structural invariants are both correct realisations of the specification.

This shifts the verification question from 'does the output match an expected form?' to 'does the output satisfy the specified invariants?' The former is impossible to answer reliably for a stochastic generator. The latter is precisely the question that tests answer.

2.3 Why This Theorem Is Foundational

Every significant shift in how systems are built has rested on a formalisable property that made the approach defensible to parties who were not already convinced. Relational databases had Codd's theorems on normal forms. TCP/IP had the end-to-end argument. Design by contract had Hoare logic. In each case, the formalisable property was not the point of the framework — the framework was the point. But without the property, the framework was indistinguishable from preference.

The Bounded Stochasticity Theorem converts SDPF from a set of useful practices into a discipline. A certification body can certify conformance to the theorem's scope conditions. A regulator can cite the

theorem as the basis for an audit trail's sufficiency. A tool vendor can build verification tooling against the invariant set. An independent implementer can check their implementation against the theorem's failure conditions and know where their tool is sound and where it is not.

> *Without the theorem, SDPF is a disciplined practice. With it, SDPF is a standard.*

2.4 Scope Conditions

The theorem does not hold unconditionally. Three conditions must be satisfied:

- The specification must be complete — missing sections leave invariants undefined.
- The TVG must be completed — unverified technical facts introduce factual errors that the theorem cannot bound.
- The AI model must have sufficient capability to satisfy the specified invariants.

These scope conditions are features, not limitations. A theorem whose scope is everything is a theorem whose scope is nothing. Precisely defined conditions tell implementers exactly when they can rely on the theorem and exactly when they cannot.

3. The Software Development Prompting Framework

SDPF is a formal specification language that makes complete specifications possible. It defines the vocabulary, grammar, semantics, and enforcement mechanisms that eliminate ambiguity at the source. A specification written in SDPF gives an AI model nothing to speculate about. The output is not a better guess — it is a mechanical execution of a verified contract.

3.1 Three Core Principles

	Principle	Definition
I	**Specification First**	No implementation begins until the contract is complete, locked, and TVG-verified. Completeness is defined structurally — all required sections present, all requirements tagged, all TVG entries passed.
II	**Facts Before Execution**	Every asserted technical fact — version numbers, API endpoints, file paths, environment variables — is verified against live output before implementation begins. This is the Technical Verification Gate.
III	**Verification Always**	No release without a complete Verification Closure Record. Eleven structural invariant checks must all pass. CLOSURE STATUS must be COMPLETE.

3.2 The Technical Verification Gate

The TVG is Core Principle II and a mandatory top-level section of every SDPF-conforming specification. It lists every tool version, library version, API endpoint, environment variable, flag, path, and configuration value asserted as fact anywhere in the specification. For each, it provides a verification command executable in the target environment and a pass condition defining the expected output.

The TVG is not a post-implementation check. It runs before any code is generated. Its purpose is to ensure that the AI receives a specification where every stated technical fact is true in the actual environment the implementation will run in.

REAL-WORLD FINDING: During the Industrial Controller PSR reference implementation, the TVG caught a critical environmental mismatch. Every draft produced without TVG verification asserted Python 3.11 on Linux. The actual environment was Python 3.13.1 on Windows 11. Every path and version number in those drafts was wrong. The TVG stopped this before implementation began — at zero cost. Discovery at deployment time would have invalidated the entire codebase.

3.3 The SDPF Lifecycle

The lifecycle enforces a strict sequence. Phase 0 — problem identification — must be complete before Phase 1 begins. Within Phase 1, each stage gates the next. The gate sequence is enforced in code, not policy. Violations produce hard errors, not advisory warnings.

Phase	Stage	Gate Condition	Output
0	Problem Definition	Validated problem statement. All 4 tests pass. Problem owner named.	Phase 0 record
1	SPEC_LOCKED	All sections present. TVG verified. Zero CRITICAL-CRITICAL conflicts.	Locked specification
1	TESTS_LOCKED	Tests derived from locked spec. All REQ-IDs covered. Tests frozen.	Locked test suite
1	CODE_GENERATED	Implementation from locked tests. Every function REQ-ID annotated.	Annotated codebase
1	VERIFIED	All 11 structural invariant checks pass.	Closure record
1	EVIDENCED	Signed evidence package exported and archived.	Tamper-evident audit trail

3.4 The 17 SDPF Styles

SDPF defines 17 formally specified styles — dialects governing semantic emphasis, required sections, verification focus, and AI framing for specific classes of system. Style selection is the first architectural decision in Phase 1, made before any requirement is written.

ID	Style	High-Assurance Relevance
8	Constraint-Based	Embedded systems, real-time systems. Hard constraints are primary requirements. Violation is a CRITICAL failure.
10	Compliance-Driven	Healthcare, finance, government. Regulatory mapping and audit evidence are built-in outputs. Every CRITICAL requirement maps to a regulatory clause.
12	Sovereign Structural Ledger	Immutable audit ledgers, chain-of-custody. Tamper evidence and provenance are CRITICAL. Mutation is a specification violation.
14	Dynamic Criticality Extension	Safety-critical systems with runtime state-dependent escalation. All conflict resolution is automated. No human escalation paths at runtime.
15	Cross-System Verification	Formal verification systems. Invariants and closure checks required at all boundaries. Proof generation is a first-class output.

3.5 The Evidence Package

Every verified SDPF run produces a signed evidence package — a structured JSON document containing the validated problem statement, locked specification, complete test suite, implementation hash, traceability matrix, verification closure record, and a cryptographic provenance signature computed using HMAC-SHA256.

Any modification to the package after signing invalidates the signature, making the evidence tamper-evident. This is not documentation produced after the fact. It is an auditable proof that the three core principles were satisfied, produced as a natural output of following the process correctly.

4. The Post-Symbolic Runtime (PSR)

4.1 Definition

The Post-Symbolic Runtime is a formally defined architectural pattern for mission-persistent autonomous systems — systems that must continue operating through component failures, power interruptions, network losses, and environmental variations without human intervention for recoverable conditions.

FORMAL DEFINITION: PSR = (S, M, P, T, A, R) where: S = State Space, M = Mission Function, P = Persistence Function, T = Transition Rules, A = Adaptation Engine, R = Resilience Function

4.2 The Six Components

	Component	Definition and High-Assurance Significance
S	**State Space**	Complete observable and internal state of the system at any point in time. In high-assurance systems, the state space must be fully enumerable — every reachable state must be defined. Undefined states are safety hazards.
M	**Mission Function**	The persistent goal the system maintains. M: States → States. Unlike classical programs that map input to output, M maintains a target condition — temperature at setpoint, vehicle on safe trajectory, patient receiving correct dosage.
P	**Persistence Function**	Save/restore across interruptions. P(s) → {continue, adapt, terminate}. State is written atomically with cryptographic signing. On restart, last valid signed state is restored. Power loss does not mean mission loss.
T	**Transition Rules**	Deterministic rules governing state transitions. In SDPF Style 14, these are the criticality escalation and de-escalation rules — structurally defined in the specification, not improvised at runtime.
A	**Adaptation Engine**	Component failover and substitution. Primary sensor fails → switch to backup within one cycle. Primary actuator fails → transfer last output value to backup. The mission continues through component loss.

R	**Resilience Function**	Failure recovery and mission continuation. Watchdog monitors heartbeat. On timeout, restore last valid state and restart mission loop. Lethal conditions activate safe state immediately. No failure mode leaves the system in an undefined state.

4.3 What PSR Is Not

PSR is not symbolic AI — it does not operate on handcrafted logic trees that break when the world presents an unanticipated situation. PSR is not a reactive system — it does not merely respond to events and return to idle. PSR is not a conventional PLC — it does not halt on failure and wait for human intervention.

PSR is post-symbolic in the precise sense: it operates beyond the assumption that a finite set of handcrafted rules can anticipate every operational condition. The mission function defines what must be achieved. The adaptation and resilience functions define how to continue achieving it through conditions the specification could not fully anticipate. The persistence function ensures that what was known before a failure is known again after recovery.

4.4 The Dynamic Criticality Model

The most significant architectural feature of PSR for high-assurance applications is the dynamic criticality model. The system's permitted response set changes at runtime based on measured process state. This is not a design choice — it is a safety requirement. A system that responds identically at NOMINAL and EMERGENCY criticality is not a safe system.

Level	Name	Autonomous Response	Operator Alert	Auth Required	Trigger Condition
0	NOMINAL	Full autonomous	None	No	All readings within target ± tolerance
1	ADVISORY	Full autonomous	Yes	No	Reading outside tolerance — trend concerning
2	ELEVATED	Reduced range	Escalated	No	Reading approaching limit or component failure
3	CRITICAL	Suspended	Urgent	Yes — for resume	Reading at safety threshold

4	EMERGENCY	Safe position	Emergency	Yes — for reset	Reading exceeds safety limit

Crucially: de-escalation from EMERGENCY requires manual operator reset with a valid authorisation token. No automatic de-escalation from EMERGENCY is permitted under any circumstances. The system cannot talk itself out of an emergency state. This is not a software policy — it is a structural requirement enforced in code.

5. SDPF and PSR: A Unified Methodology

SDPF and PSR address the same problem at different layers. SDPF closes the gap between intent and implementation — ensuring the system does exactly what was specified. PSR closes the gap between specification and operation — ensuring the system persists through the real-world failures that no specification fully anticipates.

Layer	SDPF Contribution	PSR Contribution
Specification	Complete, locked, TVG-verified contract before any code exists	Formal tuple definition ensures all six components are specified
Verification	11 structural invariant checks. Traceability matrix. Signed evidence.	Every failure mode has a defined, tested response in the specification
Operation	Every function REQ-ID annotated. Change protocol enforced.	Mission persists through sensor failure, actuator failure, power loss
Audit	Tamper-evident HMAC-signed evidence package from problem statement to verified code	HMAC-signed state persistence. Append-only signed audit log on every event.
Safety	CRITICAL requirements directly address the problem gap. TVG prevents environmental mismatches.	Dynamic criticality gates response. EMERGENCY requires human reset. Safe state is always reachable.

Together, SDPF and PSR describe a complete chain from problem statement to running mission-persistent system with a tamper-evident audit trail at every step. That chain does not exist as a unified methodology anywhere else.

5.1 The Key Insight

The key insight of the combined methodology is this: fix everything before running code.

Every hour spent fixing wrong code is an hour that should have been spent writing the specification correctly. The code was always going to be generated. The question is whether it was generated against a complete, verified contract or against a guess. When the specification is complete, the TVG passes, and the tests are locked before implementation begins — there is nothing to fix. The first verified submission is correct.

In high-assurance domains where wrong code costs lives, this is not a productivity improvement. It is a safety requirement.

6. Reference Implementation: Industrial Controller PSR

The Industrial Controller PSR was built as the complete reference implementation of the SDPF + PSR methodology. It demonstrates the full lifecycle from Phase 0 through to verified, tested, deployable code for a mission-critical industrial process controller.

6.1 System Capabilities

- Continuous non-terminating mission loop — runs until manually terminated or lethal condition
- Five-level dynamic criticality classification with structurally gated response sets
- Automatic primary-to-backup failover for sensors and actuators within one control cycle
- Atomic HMAC-SHA256 signed state persistence every 10 seconds and on every criticality transition
- Full state restoration from last valid signed record on restart — zero state loss
- Watchdog monitoring with automatic loop restart and state restoration
- SCADA reporting with queue-and-flush on network failure — mission never halts for SCADA
- Configurable for any process type: temperature, pressure, flow, level, pH, conductivity
- Append-only signed audit log on every event, transition, alert, and operator command

6.2 Lifecycle Results

Stage	Result	Key Evidence
Phase 0	**COMPLETE**	Problem statement: 100% state loss on every restart. Four validation tests passed. Owner confirmed.
TVG	**COMPLETE**	16 checks. Real finding: Python 3.13.1 Windows 11. Error caught before implementation at zero cost.
SPEC_LOCKED	**COMPLETE**	Style 14. 27 requirements. 15 CRITICAL. Zero CRITICAL-CRITICAL conflicts.

TESTS_LOCKED	**COMPLETE**	56 tests. 10 classes. All 27 REQ-IDs covered. Locked before implementation.
CODE_GENERATED	**COMPLETE**	10 modules. Every function REQ-ID annotated. No code without a requirement.
VERIFIED	**56 / 56**	All 11 structural invariant checks passed. CLOSURE STATUS: COMPLETE.

6.3 Observed Behaviour Without Hardware

When the controller ran with no physical hardware connected, it demonstrated the PSR architecture in action. With all sensors unavailable, it correctly classified the situation as ELEVATED criticality, suspended PID output safely, held last known actuator position, queued SCADA reports, continued monitoring, and maintained audit log entries — all without human intervention. The mission persisted in a degraded but safe state, exactly as specified.

This is the definitive test of a PSR: it does the right thing not when everything works, but when nothing does.

7. High-Assurance Domain Applications

The PSR + SDPF methodology addresses the specific structural requirements of four high-assurance domains. In each case, the fit is not coincidental — it is structural. The domains require precisely what the methodology produces: complete requirements traceability, verified behaviour, tamper-evident audit trails, and systems that survive their operational environment.

7.1 Autonomous Vehicles

The PSR Architecture Fit

An autonomous vehicle is a PSR operating in the most demanding environment imaginable. Sensors fail. Actuators degrade. Network connections drop. The mission — navigate safely from origin to destination — must persist through all of these. Style 14 Dynamic Criticality is the natural fit.

Criticality	Vehicle Behaviour
NOMINAL	All sensors functioning. Full autonomous operation within defined operating domain.
ADVISORY	Single sensor degraded. Confidence interval widened. Speed reduced. Following distance increased.
ELEVATED	Multiple sensors degraded or weather limit approached. Conservative mode. Driver notification.
CRITICAL	Sensor fusion below safety threshold. Request driver takeover. Prepare controlled handover.
EMERGENCY	Imminent collision or total sensor loss. Minimum risk manoeuvre. Controlled stop.

Regulatory Alignment

ISO 26262 (Road Vehicle Functional Safety) requires hazard analysis, safety goal definition, requirements traceability from safety goals to implementation, and verification evidence. SDPF's Phase 0 problem statement maps to hazard analysis. CRITICAL requirements map to safety goals. The traceability matrix satisfies requirements coverage. The signed evidence package satisfies verification

evidence requirements. ASIL-D compliance — the highest functional safety integrity level — requires exactly the documentation discipline that SDPF produces automatically.

7.2 Spacecraft

The PSR Architecture Fit

Spacecraft are the original PSR. The Voyager probes have operated for nearly fifty years, adapting to hardware failures, restoring state after cosmic ray upsets, and continuing their mission with components their designers did not anticipate losing. Every deep space mission operates on a version of the PSR pattern because there is no alternative when the round-trip communication delay is measured in hours.

The critical PSR property for spacecraft is the persistence function. A cosmic ray event can corrupt processor state, reset a microcontroller, or cause a soft error in memory. A spacecraft PSR must restore its mission state from the last valid signed checkpoint and resume without ground intervention. The mission cannot wait for a communications window.

> **MISSION PERSISTENCE:** The PSR Resilience Function — watchdog detection, state restoration, mission resumption — is precisely the architecture that spacecraft fault protection systems implement. SDPF provides the formal specification discipline to specify, verify, and certify these systems to the standard that space agencies require.

Regulatory Alignment

NASA NPR 7150.2 (Software Engineering Requirements) requires software classification, requirements traceability, independent verification and validation, and documented evidence of compliance for Class C (critical) and Class D (safety-critical) software. SDPF produces all of this as a natural output. The cost reduction for a methodology that generates space-grade software evidence automatically — rather than through enormously expensive manual documentation processes — would be substantial.

7.3 Power Grids

The PSR Architecture Fit

Power grid control systems must maintain continuous operation through equipment failures, switch to backup systems automatically, and maintain safe state during faults — all without interruption to supply. This is the PSR mission function: maintain process (frequency, voltage, load balance) within target ± tolerance, continuously, through every recoverable failure.

The dynamic criticality model maps directly to grid operating states. NOMINAL is normal operation. ADVISORY is N-1 contingency — the grid can survive the loss of one more element. ELEVATED is N-2 contingency — further loss would be serious. CRITICAL is emergency — load shedding may be required. EMERGENCY is imminent blackout — all available interventions activated.

Grid State	PSR Response
Normal (N)	Full autonomous frequency and voltage regulation. Continuous monitoring.
Alert (N-1)	Advisory alert. Operator notified. Automated reserve activation within approved bounds.
Emergency (N-2)	Elevated alert. Automatic load balancing. Operator authorisation required for major switching.
Extreme (N-3)	Critical alert. Controlled load shedding within pre-approved bounds. All actions require authorisation.
Blackout prevention	Emergency safe state. All pre-approved automatic interventions. Manual reset to resume normal.

Regulatory Alignment

NERC CIP (Critical Infrastructure Protection) standards require configuration change management, electronic security perimeter controls, and incident response planning for Bulk Electric System cyber systems. SDPF's change protocol — classifying every post-lock modification as BREAKING, SIGNIFICANT, or MINOR with mandatory documentation — satisfies CIP-010 configuration change management requirements. The signed evidence package satisfies CIP-007 system security management logging requirements.

7.4 Medical Equipment

The PSR Architecture Fit

For life-sustaining medical devices — ventilators, infusion pumps, implantable devices, patient monitoring systems — the PSR mission function is the patient's life. The mission must persist through sensor failures, power interruptions, communication losses, and software errors. A ventilator that halts when a flow sensor fails is not acceptable. A ventilator PSR that detects the failure, switches to backup sensing, escalates criticality, alerts clinical staff, and continues delivering breaths within a conservatively defined safe envelope is what the patient requires.

The criticality model for a ventilator illustrates the architecture:

- NOMINAL: All sensors functioning. Patient parameters within prescribed range. Autonomous delivery.
- ADVISORY: Single sensor degraded. Backup active. Clinical notification. Conservative delivery parameters.
- ELEVATED: Patient parameter outside prescribed range. Alarm active. Delivery adjusted to safe bounds.
- CRITICAL: Multiple system failures. Backup power active. Clinical team notified urgently. Manual override available.
- EMERGENCY: Life-threatening condition detected. Maximum safe intervention. Immediate clinical response required.

Regulatory Alignment

IEC 62304 (Medical Device Software Life Cycle Processes) defines three software safety classes. Class C — failure may result in death or serious injury — requires complete requirements traceability, verified testing at unit, integration, and system levels, and documented evidence of compliance with the full software development lifecycle.

SDPF Style 10 (Compliance-Driven) was designed for this context. Every CRITICAL requirement carries a regulatory mapping. The audit evidence requirements are built into the specification before any code is written. IEC 60601-1 (Medical Electrical Equipment) safety requirements map directly to CRITICAL requirements in the SDPF specification. The evidence package satisfies the technical file requirements for CE marking and FDA 510(k) submission.

8. Regulatory Alignment

SDPF produces regulatory compliance as a by-product of good engineering practice — not as an additional documentation burden. The following table maps SDPF outputs to the requirements of major high-assurance regulatory frameworks.

Standard	Domain	SDPF Satisfaction
DO-178C	Avionics	Requirements traceability (SDPF traceability matrix). Independence of testing (tests derived from spec, not code). Verification evidence (signed closure record). Configuration management (change protocol).
ISO 26262	Automotive	Hazard analysis (Phase 0 problem statement). Safety goal traceability (CRITICAL requirements). ASIL allocation (criticality classification). Verification coverage (56-test suite from locked spec).
IEC 62304	Medical devices	Software development lifecycle (full SDPF lifecycle). Requirements traceability (REQ-ID to test to implementation). Risk management integration (CRITICAL requirements map to hazards). Class C compliance (SDPF Style 10).
IEC 61508	Functional safety	SIL determination (criticality classification). Requirements specification (locked SDPF spec). Verification (11 structural invariant checks). Functional safety management (evidence package).
NERC CIP	Power grids	Configuration change management (SDPF change protocol). Security controls documentation (TVG + evidence package). Incident response (PSR resilience function + audit log).
NASA NPR 7150.2	Spacecraft	Software classification (SDPF style selection). Requirements traceability (traceability matrix). IV&V evidence (signed closure record). Class C/D compliance (SDPF styles 10, 14, 15).
FDA 21 CFR Part 11	Medical records	Electronic record integrity (HMAC-signed evidence package). Audit trail (append-only signed log). Access

		controls (authorisation gate). Computer system validation (SDPF verification gate).

8.1 The Safety Case Argument

In high-assurance engineering, a safety case is a structured argument — supported by evidence — that a system is acceptably safe for a specific application in a specific environment. Building a safety case is typically one of the most expensive parts of certifying a high-assurance system, often consuming as much effort as the development itself.

SDPF does not replace the safety argument — the human engineer still makes the judgement that the requirements are correct and the hazard analysis is complete. But SDPF produces the evidence that supports the argument automatically, as a natural output of following the process.

Safety Case Element	SDPF Evidence Produced
Hazard identification	Phase 0 problem statement — observable, bounded, cause-free, solution-free gap analysis
Safety requirements	CRITICAL requirements — every one traces directly to closing the Phase 0 gap
Requirements coverage	Traceability matrix — REQ-ID to TEST-ID to implementation module
Verification evidence	Signed verification closure record — 11 checks, CLOSURE STATUS: COMPLETE
Configuration management	BREAKING/SIGNIFICANT/MINOR change protocol — every post-lock change classified and documented
Audit trail integrity	HMAC-SHA256 signed evidence package — tamper-evident from problem statement to verified code
Environmental verification	TVG — every technical fact verified against live output before implementation begins

9. The Time and Cost Argument

9.1 Where Time Is Actually Lost

The industry measures development time as time spent writing code. This is the wrong measurement. The expensive part of software development is not writing code — it is writing the wrong code and fixing it. Correction cycles. Debugging sessions where the root cause turns out to be an assumption that was never validated. Rework when the implementation does not match what was wanted because what was wanted was never precisely stated.

In high-assurance domains, the cost of wrong code is compounded by the cost of re-certification. A change to a safety-critical system that requires re-running DO-178C verification, re-submitting an FDA 510(k), or re-certifying to IEC 61508 is not measured in developer-hours. It is measured in months and millions of dollars.

9.2 Measured Outcome

The Industrial Controller PSR — a 10-module mission-critical system with dynamic criticality classification, component failover, HMAC-signed persistence, watchdog recovery, and SCADA reporting — was produced from problem statement to 56 passing verified tests in hours.

Activity	SDPF Duration	Conventional Estimate
Phase 0 — problem definition	One session	1-2 weeks of meetings, documentation, review
TVG — environment verification	One session	Discovered at build/deploy — days of debugging
Specification — 27 requirements	One session	2-4 weeks of requirements engineering
Test suite — 56 tests	One session	1-3 weeks (typically written after code)
Implementation — 10 modules	One session	4-8 weeks of development
Verification — 56/56	One session + single	Weeks of debugging, correction cycles,

	fix	regression
Evidence package	Automatic output	Weeks of manual documentation assembly
TOTAL	**Hours**	**3-6 months**

9.3 The High-Assurance Multiplier

In standard software development, the cost of a specification error discovered at implementation time is measured in developer-days. In high-assurance domains, the multiplier is much larger.

Domain	Error Found At	Consequence
Avionics	Post-certification	Full DO-178C re-certification. Potentially $10M+ and 12-18 months.
Medical device	Post-510(k) clearance	FDA recall, re-submission, clinical hold. 2-5 year impact.
Automotive	Post-ASIL certification	ISO 26262 re-certification. Potential recall. Liability exposure.
Power grid	Post-deployment	NERC CIP violation. Potential grid incident. Regulatory sanction.
Spacecraft	Post-launch	Mission loss. No recovery. Cost: $100M - $10B+.

The TVG alone — which costs one session to complete and catches environmental errors at specification time — eliminates the category of failure where a correctly specified, correctly implemented system fails in the target environment because the environment was different from what the specification assumed. In high-assurance domains, that category of failure triggers the consequences in the table above.

10. Conclusions and Implications

10.1 What Has Been Established

This white paper has presented the Post-Symbolic Runtime and the Software Development Prompting Framework as a unified formal methodology for high-assurance autonomous systems. The following has been established:

1. The Bounded Stochasticity Theorem provides the formal basis for deterministic specifications governing stochastic AI generators. This converts the methodology from a set of practices to a certifiable standard.
2. SDPF produces regulatory compliance evidence — requirements traceability, verification coverage, tamper-evident audit trail — as a natural output of following the development process, not as an additional documentation burden.
3. The PSR architecture provides a complete formal model for mission-persistent autonomous systems that is immediately specifiable, verifiable, and certifiable under SDPF.
4. The combined methodology maps directly to the requirements of DO-178C, ISO 26262, IEC 62304, IEC 61508, NERC CIP, NASA NPR 7150.2, and FDA 21 CFR Part 11.
5. The reference implementation — Industrial Controller PSR — achieved 56/56 tests passing against a locked specification, with a complete evidence trail from Phase 0 problem statement to signed verification closure record.
6. The time from problem statement to verified implementation is measured in hours, not months. The cost reduction for high-assurance systems — where re-certification after a specification error is measured in years and millions — is structural, not marginal.

10.2 The Central Principle

Fix everything before running code. Every hour spent fixing wrong code is an hour that should have been spent writing the specification correctly. In domains where wrong code costs lives, this is not a productivity improvement. It is a safety requirement.

— Hamza Abdullah

10.3 The Broader Implication

The combination of PSR and SDPF addresses a problem that the high-assurance industry has been treating as intractable: how do you build a system that is both precisely specified enough to certify and robustly operational enough to survive its environment?

The answer is that these are not competing requirements. A complete specification, verified against the real environment before implementation begins, produces a system that is both. SDPF provides the specification discipline. PSR provides the operational architecture. Together they produce software that is correct by construction, auditable by design, and safe to change because every change is traceable, classified, and verified.

The methodology is founded on a formal theorem, demonstrated in a working reference implementation, and aligned to the regulatory frameworks that govern the domains where it matters most. The foundations are correctly laid.

Problem first. Specification second. Facts before execution. Verification always.

SDPF Language Specification v1.3.1 · Post-Symbolic Runtime · Hamza Abdullah · 2026

References

Abdullah, H. (2026). SDPF Language Specification, Version 1.3.1. Software Development Prompting Framework.

Abdullah, H. (2026). The Bounded Stochasticity Theorem: Why It Is Relevant, Unique, and Important.

Abdullah, H. (2026). The Technical Verification Gate: Where the Framework Meets Reality.

Abdullah, H. (2026). SDPF: A New Science of Specification for Stochastic Generators.

RTCA DO-178C (2011). Software Considerations in Airborne Systems and Equipment Certification.

ISO 26262 (2018). Road vehicles — Functional safety.

IEC 62304 (2015). Medical device software — Software life cycle processes.

IEC 61508 (2010). Functional Safety of E/E/PE Safety-related Systems.

NERC CIP Standards (2024). Critical Infrastructure Protection Standards for the Bulk Electric System.

NASA NPR 7150.2D (2022). NASA Software Engineering Requirements.

IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering.

NIST AI RMF 1.0 (2023). Artificial Intelligence Risk Management Framework.